

コマ2: AST + インタープリター

今日のゴール

提供 Lexer/Parser が返す AST を走査し、`eval_ast()` で式を評価する。

この回では RV64 アセンブリは出力しない。C プログラム全体を実行するのではなく、`main` 関数の中にある `return 式;` の「式」だけを評価する。

目的は、AST を「木」として読み、再帰的に処理する感覚を掴むことである。

1 この回で扱う範囲

対象にするプログラムは、次の形に限定する。

```
int main() {  
    return 式;  
}
```

扱う式は以下の通り。

種類	例
整数リテラル	42
単項マイナス	-3
足し算	1 + 2
引き算	10 - 3
掛け算	2 * 3
割り算	10 / 2
剰余	10 % 3
カッコ	(1 + 2) * 3

変数、代入、if、while、関数呼び出しはまだ扱わない。

2 AST を見てみる

まず、提供 Parser がどのような AST を返すか確認する。

```
python3 scaffold/parse_viewer.py sessions/02_interpreter/tests/add_mul.c
```

このプログラムの内容は次の通り。

```
int main() {  
    return 1 + 2 * 3;  
}
```

実行すると、次のような S 式が表示される。

```
(program  
  (funcdef "main" :type int (params)  
    (block  
      (return  
        (add (num 1)  
              (mul (num 2) (num 3)))))))
```

この出力は、`1 + 2 * 3` が次の構造として解析されたことを表している。

S 式の部分	意味
(program ...)	プログラム全体
(funcdef "main" ...)	main 関数の定義
(block ...)	関数本体の { ... }
(return ...)	return 文
(add A B)	A + B
(mul A B)	A * B
(num 1)	整数リテラル 1

つまり、`1 + 2 * 3` は次のような木として表されている。

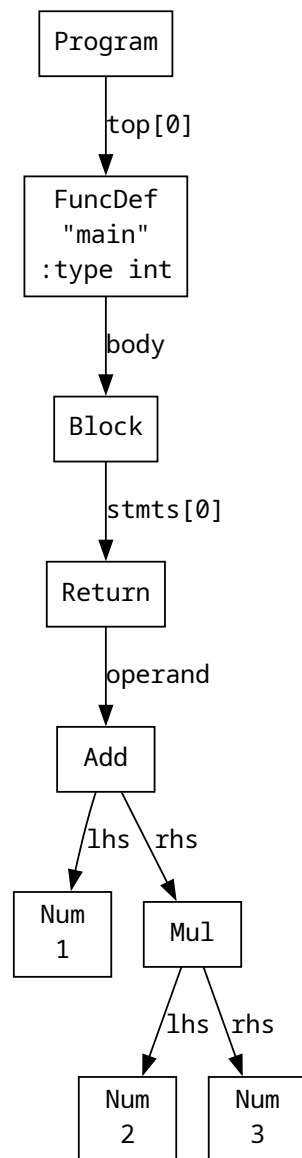


図1 add_mul.c のAST

```

(add
  (num 1)
  (mul (num 2) (num 3)))
    
```

2 * 3 が add の右側の子になっている。演算子の優先順位は Parser がすでに処理しているため、eval_ast() は渡された木をそのまま評価すればよい。

3 Node の構造

`parse_viewer.py` の S 式は表示用の形式である。`mycc.py` に渡される実体は、`scaffold/ast_def.py` で定義されている `Node` オブジェクトである。

`Node` は、AST の 1 つの節点を表す入れ物である。たとえば、数値、足し算、掛け算は、それぞれ別の種類の `Node` として表される。

主に見るフィールドは次の通り。

フィールド	意味
<code>node.kind</code>	ノードの種類
<code>node.val</code>	整数リテラルの値
<code>node.lhs</code>	二項演算の左側の子ノード
<code>node.rhs</code>	二項演算の右側の子ノード
<code>node.operand</code>	単項演算の対象ノード

ノードの種類は、`ast_def.py` で定義されている定数で判定する。

S 式	<code>Node</code> での表現
<code>(num 42)</code>	<code>node.kind == ND_NUM, node.val == 42</code>
<code>(neg A)</code>	<code>node.kind == ND_NEG, node.operand == A</code>
<code>(add A B)</code>	<code>node.kind == ND_ADD, node.lhs == A, node.rhs == B</code>
<code>(sub A B)</code>	<code>node.kind == ND_SUB, node.lhs == A, node.rhs == B</code>
<code>(mul A B)</code>	<code>node.kind == ND_MUL, node.lhs == A, node.rhs == B</code>
<code>(div A B)</code>	<code>node.kind == ND_DIV, node.lhs == A, node.rhs == B</code>
<code>(mod A B)</code>	<code>node.kind == ND_MOD, node.lhs == A, node.rhs == B</code>

たとえば、次の S 式を考える。

```
(add (num 1) (mul (num 2) (num 3)))
```

これは `Node` では次のような関係になっている。

場所	内容
一番上のノード	<code>kind == ND_ADD</code>
<code>ND_ADD</code> の lhs	<code>ND_NUM, val == 1</code>
<code>ND_ADD</code> の rhs	<code>ND_MUL</code>
<code>ND_MUL</code> の lhs	<code>ND_NUM, val == 2</code>
<code>ND_MUL</code> の rhs	<code>ND_NUM, val == 3</code>

4 実装方針

`eval_ast(node)` は、`node.kind` を見て処理を分ける。`node.kind` は文字列（`'Num'`, `'Add'`, ...）であり、`ast_def.py` で定数 `ND_NUM`, `ND_ADD` 等も定義されている。スケルトンでは `eval_ast` が `TODO` になっているため、以下の各ケースを実装する。

方針は次の通り。

ノード種別	評価方法
<code>'Num' (ND_NUM)</code>	<code>node.val</code> を返す
<code>'Neg' (ND_NEG)</code>	<code>node.operand</code> を評価し、符号を反転する
<code>'Add' (ND_ADD)</code>	<code>node.lhs</code> と <code>node.rhs</code> を評価し、足す
<code>'Sub' (ND_SUB)</code>	<code>node.lhs</code> と <code>node.rhs</code> を評価し、引く
<code>'Mul' (ND_MUL)</code>	<code>node.lhs</code> と <code>node.rhs</code> を評価し、掛ける
<code>'Div' (ND_DIV)</code>	<code>node.lhs</code> と <code>node.rhs</code> を評価し、割る
<code>'Mod' (ND_MOD)</code>	<code>node.lhs</code> と <code>node.rhs</code> を評価し、剰余を求める

重要なのは、子ノードもまた `Node` であるという点である。そのため、子ノードの値を求めるときも同じ `eval_ast()` を使える。

このように、自分自身を使って木をたどる関数を再帰関数という。

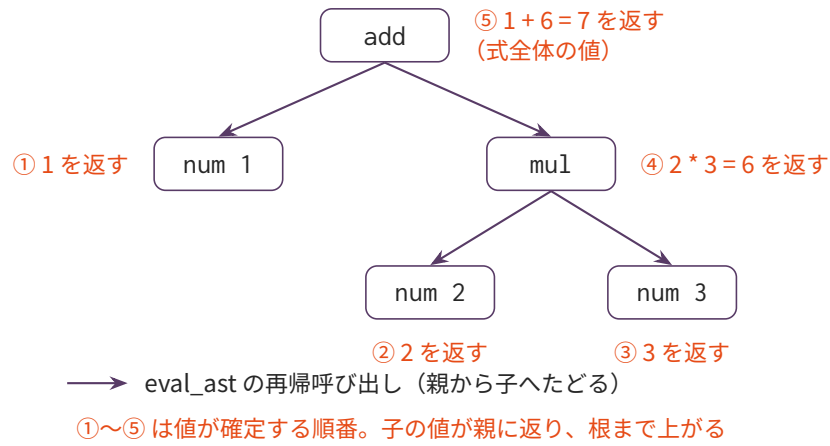


図2 eval_ast が $1 + 2 * 3$ の AST を評価する流れ

葉の値が親に返り、根まで上がると式全体の値になる。①～⑤は値が確定する順番である。

5 編集するファイル

- mycc.py

eval_ast(node) を実装する。

6 tests/

ファイル	内容	期待値
add_mul.c	$1 + 2 * 3$	7
div_mod.c	$100 / 4 + 17 \% 5$	27
paren.c	$(10 - 3) * 2$	14
neg.c	$-3 + 10$	7

7 テスト

```
python3 sessions/02_interpreter/check.py
```

`tests/*.c` の `main` 関数内の `return` 式を評価し、対応する `.ans` と比較する。

個別に実行する場合は、次のようにする。

```
python3 sessions/02_interpreter/mycc.py sessions/02_interpreter/tests/add_mul.c
```

成功すると、次のように表示される。

```
評価結果: 7
```

8 注意

この回のテストでは、割り算・剰余の右辺は 0 にならない。

また、この回では負数を含む割り算・剰余は扱わない。Python の `//` や `%` は、負数を含む場合に C の整数除算・剰余と挙動が異なるためである。